SYNESIS 26 · FINTECH · PROBLEM 2

InvoiceTrust

Verified invoice financing with a tamper-evident audit trail

Product & Build Guide

How real platforms work · What we build · How we build it in 6 hours · How we stand out



Change one link and the whole chain shows exactly where it broke.

100% free stack **Android APK ready** **6-hour build plan**

Working name. Prototype only: mock data, no real money moves.

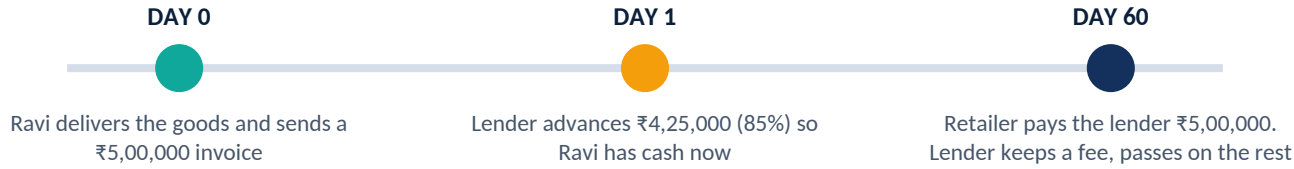
13 October 2026

1 The problem, in plain words

Why invoice financing exists, and why lenders are nervous

Meet Ravi. He runs a small factory making plastic buckets. He delivers goods worth ₹5,00,000 to a big retailer, who says: "We will pay in 60 days." But Ravi must pay his workers and buy raw material **this week**. A bank-style loan is slow. So he asks a lender for cash against the unpaid invoice.

Invoice financing in one picture



The lender is handing over money because of a piece of paper. That paper is the whole risk. Before paying, the lender needs answers to four questions:

Is the invoice genuine?
Not faked, not edited, not made up by the supplier.

Was it really delivered?
Goods or services reached the buyer, in the quantity billed.

Will the buyer pay?
Look at the buyer's history of paying on time, not the supplier's promise.

Is it financed elsewhere?
The same invoice must not be sold to two lenders (double financing).

What goes wrong today

Documents live in different places (invoice, purchase order, delivery receipt, contract) and people check them by hand. That causes **duplicate financing, manipulated invoices, slow approvals and disputes**. The problem statement asks us for a platform that automates this verification and keeps a history nobody can quietly rewrite.

Our answer in one sentence

A checker that **reads the invoice, tests it against several kinds of evidence, explains its decision in plain English, and records every step in a tamper-evident log**, while keeping the actual document private.

2 How real platforms work

The real-world version of what we are prototyping

In India the Reserve Bank of India set up **TReDS** (Trade Receivables Discounting System): regulated online marketplaces where small suppliers (MSMEs) list invoices raised on large buyers, and banks and non-bank lenders bid to buy those invoices at a discount. Licensed platforms include RXIL, Invoicemart (A.TReDS) and M1xchange, with C2treds also listed among them. On these platforms an invoice is called a **"factoring unit"**, and the discounting is generally without recourse to the supplier.

The standard flow

1. **Supplier delivers** — Ships the goods and raises an invoice on the buyer.
2. **Invoice listed** — Uploaded to the platform as a "factoring unit".
3. **Buyer confirms** — Buyer accepts the invoice within a set time window.
4. **Financiers bid** — Banks and NBFCs offer to buy it at a discount; supplier gets cash early.
5. **Buyer pays later** — On the due date the buyer settles, and the financier is repaid.

Real platform vs our prototype

Aspect	Real TReDS-style platform	Our prototype
People involved	MSME seller, large corporate buyer, banks and NBFCs as financiers	Four mock roles: supplier, buyer, lender, auditor
Invoice entry	Seller registers the invoice as a factoring unit	Supplier uploads a photo or PDF; AI reads the fields
Buyer's role	Accepts the invoice within a time window	Confirms or disputes with one tap
Financing	Financiers bid; settlement is on the platform	One mock lender approves with a risk-based advance (stretch: 2 to 3 lenders compare offers)
Trust and records	Regulated and licensed by the RBI	Hash-chained audit log. Not licensed, no real money, mock data

Where the industry is heading

Industry commentary lists **AI-based invoice verification** among the directions the market is moving. That is exactly the gap this problem statement points at: replace manual document checks with automated, explainable ones.

Facts in this section come from the public sources listed on the last page.

3 What we are building

Users, screens and one clean core flow

Four roles, one story

Role	Who they are	What they do in the app
Supplier	Small business that wants cash early	Uploads invoices, confirms the extracted fields, sees status and the money offered
Buyer	Company that owes the money	Sees invoices raised on them and confirms or disputes each one (a very small screen)
Lender	Finance officer at the bank or NBFC	Sees a ranked queue with trust score and reasons; approves, rejects or reviews flagged cases
Auditor	Admin or compliance person	Opens the audit chain and presses **Verify chain** to prove nothing was altered

A walk through the app

1. **Supplier** logs in on the phone, taps Upload and photographs the invoice.
2. The app shows the **fields the AI extracted** (invoice number, buyer, amount, items). The supplier fixes anything wrong and confirms.
3. The **engine runs the checks** in seconds and writes each step to the audit log.
4. The **buyer** sees the invoice and taps Confirm (or Dispute).
5. The **lender** sees the invoice with a trust score, plain-English reasons and a suggested advance. Clean ones are auto-approved; flagged ones wait for a human.
6. The supplier sees **"Approved: ₹2,04,000 advance"** or the exact reason it was held.
7. The **auditor** presses Verify chain. Green means untouched; red shows the exact entry that broke.

Scope for 6 hours

Must build (MVP)

Upload + AI extraction · duplicate, identity and PO/receipt checks · trust score and decision · buyer confirmation · hash-chained audit log + Verify chain · role-based login.

If time allows

Verification QR on each approved invoice · risk-based advance rate · lender dashboard charts · comparison of 2 to 3 mock lender offers · anchor the head hash on a testnet.

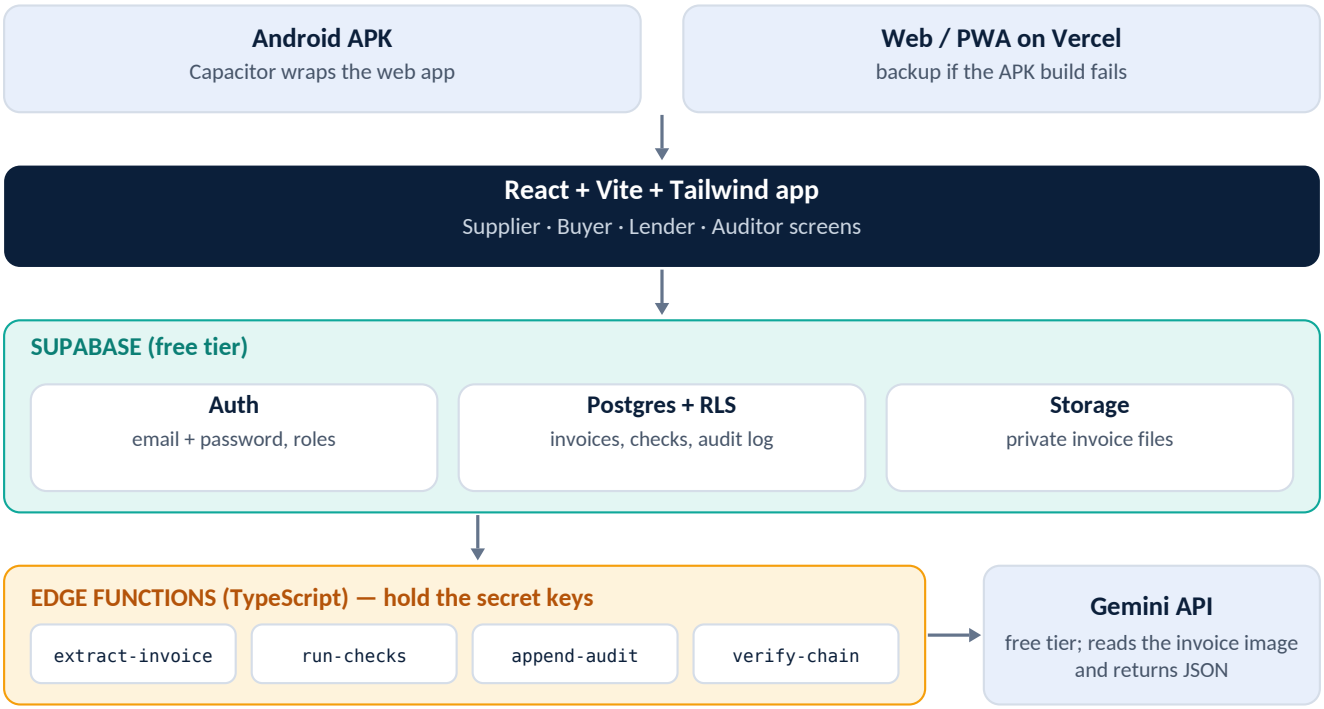
Future scope (just say it)

Real KYC and GST lookups · logistics-based delivery proof · payment tracking and late-payment handling · a regulated ledger for anchoring.

Principle: one core flow done perfectly beats ten half-working features. The flow is **Upload → Check → Decide → Record → Verify**.

4 How it is built

Architecture and the free stack



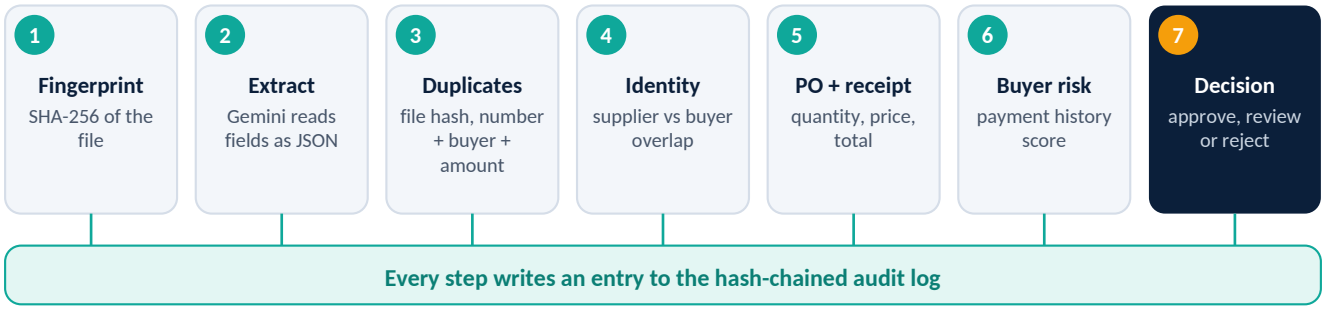
Piece	Tool (free)	Why this one
App	React + Vite + Tailwind	Fast to build in Claude Code; works as a website and inside the APK
Android APK	Capacitor	Wraps the web app; camera upload works. Test it in hour 1 (needs Android Studio)
Backend	Supabase free tier	Login, Postgres, private file storage and edge functions in one place
AI	Gemini API free tier (Google AI Studio)	Reads invoice images into JSON. Free limits change, so check them before the event
Hosting	Vercel Hobby	One-click deploy of the web build; also your PWA backup
Audit chain	A Postgres table + SHA-256	Simple and fast. Optional bonus: anchor the latest hash on a testnet

Keep secrets off the phone

The Gemini key lives only in an **edge function secret**, never in the app. The app calls our function; the function calls Gemini. If the key were in the APK, anyone could extract it.

5 The verification engine

How the app tells clean from duplicate, mismatched and tampered



Every check returns the same small answer: **pass, warn or fail, plus a reason in plain English**. The AI only *reads* the invoice. Decisions are made by ordinary code, so they are repeatable and explainable.

The five demo cases

Case	What we upload	Check that catches it	What the screen says
Clean	INV-1001, ₹2,40,000, matches PO and delivery receipt	All checks pass	**Approved**. Advance ₹2,04,000 (85%)
Duplicate	The same file again, or a re-exported copy with the same number, buyer and amount	Duplicate check (file hash, then number + buyer + amount)	**Rejected**. Already submitted on that date
Mismatched	INV-1002 bills 100 units; the delivery receipt shows 80	PO and receipt cross-check	**Manual review**. Invoice qty 100 vs delivered 80 (+25%)
Self-dealing	Supplier and buyer share one bank account number	Identity check	**Rejected**. Shared bank account, possible self-dealing
Tampered	Edit an approved invoice's amount directly in the database	Verify chain (audit screen)	**Red**. Chain breaks at the exact entry

Decision rules

Outcome	When it happens
Auto-approve	No failures or warnings, buyer has confirmed, and trust score is 70 or higher
Manual review	Any warning: quantity or price gap above 2%, possible duplicate, new buyer with no history, low AI confidence, or trust score 40 to 69
Reject	Any hard failure: exact duplicate, already financed, supplier and buyer share identity details, no matching PO or receipt, or the audit chain fails to verify

Trust score and advance rate (demo settings, tune freely)

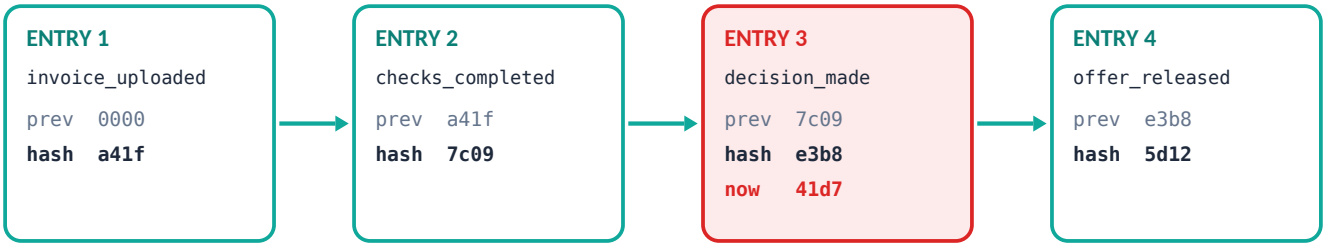
```
trust = 50 x on_time_rate
      + 25 x max(0, 1 - avg_delay_days / 30)
      + 25 x max(0, 1 - outstanding / credit_limit)      # 0 to 100
new buyer (no history)  -> score capped at 60  -> manual review
advance % : 85 for score 80+, 75 for 70 to 79
below 70   : goes to manual review (suggested advance 65)
```

6 The hybrid audit chain

Off-chain files, tamper-evident fingerprints

Hybrid means two stores with different jobs. The sensitive document stays in ordinary private storage. Only its **fingerprint** (a SHA-256 hash) goes into a log where every entry depends on the one before it. Change anything later and the fingerprints stop matching, so the log proves tampering without ever showing the document.

Off-chain (private)	Tamper-evident record (safe to expose)
The invoice file in a private Supabase bucket; full invoice data in Postgres; visible only to the right roles	Hash of the file, hash of the invoice record, event type, time, and the previous entry's hash. No names, no amounts needed to verify



How an entry is made

```
payload_hash = SHA256( canonical JSON of {invoice_id, file_hash, record_hash, summary} )
entry_hash   = SHA256( prev_hash + event_type + payload_hash + timestamp )
events: invoice_uploaded, fields_extracted, checks_completed, buyer_confirmed,
        decision_made, offer_released
```

What Verify chain checks

Someone does this	The check that catches it
Edits an audit entry	Recomputed entry hash no longer matches the stored one
Deletes or reorders an entry	The next entry's *prev_hash* link no longer matches
Changes an invoice amount in the database	Hash of the current invoice row differs from the *record_hash* saved in the latest entry
Swaps the stored file for another	Re-hashing the file differs from the *file_hash* on record

Honest limit

Someone with full control of the database could rebuild the *whole* chain consistently. The fix is to anchor the newest hash (the "head") somewhere they cannot rewrite: a public testnet transaction in the stretch version, or at minimum show the head hash on the audit screen and share it with the auditor. Say this in the pitch. Judges respect honesty about limits.

7 Nooks and corners

What can go wrong, and how we handle it

Risk	What could happen	How we handle it	When
Self-dealing	Supplier invents a buyer account to confirm its own fake invoice	Compare GSTIN, bank account, phone, email domain and address. Buyer must confirm separately. New buyers get low limits	MVP
Same file twice	Identical upload financed again	SHA-256 of the file; also a unique database constraint so two simultaneous uploads cannot both win	MVP
Re-scanned or renumbered invoice	Same bill, different file or number	Match on number + buyer + amount; warn on same buyer + amount + date with a new number	MVP
Edited after approval	Amount changed in the database	Verify chain compares current record with recorded hash	MVP
Quantity or price mismatch	Invoice bills more than ordered or delivered	Compare against PO and receipt; 2% tolerance for rounding and tax; reason shown in plain English	MVP
Missing PO or receipt	Nothing to verify against	Hard reject with a clear message	MVP
Fake delivery receipt	Supplier uploads its own proof	Receipts belong to the buyer role (seeded in the demo); logistics data later	MVP
AI misreads the invoice	Blurry photo or odd layout gives wrong numbers	Show extracted fields for the supplier to confirm; AI confidence field; required-field validation; low confidence goes to review	MVP
Prompt injection	Text inside an invoice says “approve this”	The AI only extracts fields. Decisions come from code, and extracted text is treated as data	MVP
AI quota or outage	Gemini fails during the live demo	Cache results for the sample invoices; manual entry form as fallback	MVP
Wrong file type or size	Huge or unsupported upload	Accept PDF, JPG, PNG up to 5 MB; friendly error	MVP
Partial deliveries	Several invoices against one PO	Sum of invoiced amounts must not exceed the PO total	MVP-lite
Buyer disputes later	Buyer says the goods were faulty	Dispute status pauses payout and is logged in the chain	Later
Buyer pays late or never	Lender is exposed	Update buyer payment history; recourse and collections are out of scope	Later
Privacy	Sensitive documents leak	Private bucket, 60-second signed links, only hashes in the audit log	MVP

8 Login, roles and security

Simple to build, safe to demo

Use **Supabase Auth with email and password**. Google login needs OAuth setup and redirects that behave badly inside an APK. Pre-create four demo accounts (two suppliers, one buyer, one lender, one auditor) so no demo time is wasted on sign-up. Store each user's role in a *profiles* table and send them to their own screen after login.

Who can see and do what (Row Level Security)

Role	Can read	Can write
Supplier	Only their own invoices, checks and offers	Create invoices; confirm their own extracted fields
Buyer	Invoices raised on them; their own POs and receipts	Confirm or dispute those invoices
Lender	All invoices, checks, buyer scores	Approve, reject or request review; create offers
Auditor	Everything, read-only, including the audit log	Nothing (can run Verify chain)
Nobody from the app	—	Audit log is append-only and written only by an edge function

Security checklist

- Gemini key and the Supabase service-role key are **secrets in edge functions**, never in the app code or the repository.
- Invoice files go in a **private bucket**, in a folder per supplier; lenders open them through short-lived signed links.
- **Test the denied cases**: log in as Supplier B and confirm Supplier A's invoice cannot be read or listed. Show this in the demo: "access denied" is a feature.
- The live tamper demo works because you edit the row from the Supabase dashboard, which sits outside the app's rules. That is the point: the chain notices even when the app does not.
- Use mock tax IDs and bank numbers. Never put real personal or financial data in the prototype.

> This is a prototype. No real money moves and it is not a licensed platform. Say so clearly in the pitch so no one thinks you handle actual funds.

9 The data model

Eight tables are enough

Table	Key columns	Purpose
profiles	user_id, role, party_id	Links a login to a role and a company
parties	id, type (supplier or buyer), name, gstin, bank_account, phone, email, address, credit_limit, on_time_rate, avg_delay_days, outstanding	Companies, with the identity fields used for self-dealing checks and the history used for trust score
purchase_orders	po_number, buyer_id, supplier_id, item, qty, unit_price, total	What the buyer ordered (seeded)
delivery_receipts	po_id, qty_delivered, delivered_on, confirmed_by_buyer	What was actually delivered (seeded)
invoices	invoice_no, supplier_id, buyer_id, po_number, item, qty, unit_price, amount, invoice_date, due_date, file_path, file_hash, status, buyer_confirmed, ai_confidence, trust_score, advance_pct	The core record. **UNIQUE**(supplier_id, buyer_id, invoice_no) stops race-condition duplicates
invoice_checks	invoice_id, check_name, status, reason, points	One row per check, so the screen can list reasons
financing_offers	invoice_id, lender_id, advance_amount, fee, status	What the lender offers and whether it was accepted
audit_log	id, invoice_id, event_type, payload (jsonb), payload_hash, prev_hash, entry_hash, created_at	The hash chain. Append-only

Prepare the data pack before the event

- **5 buyers** with different payment histories: one excellent, one average, one slow, one brand-new, one that shares details with a supplier.
- **POs and delivery receipts** for the clean and mismatched invoices (the mismatch is 100 units billed vs 80 delivered).
- **Five invoice files**: clean, duplicate (re-save the clean one), mismatched, self-dealing, and one clean invoice you will tamper with live. Make them look realistic: GST number, HSN code, line items, tax.
- Try one Tamil or bilingual invoice to see how well the AI reads it. If it works, it becomes a nice extra.

10 How we stand out

New features, and how to keep adding them

This is what we add on top of the basic “upload an invoice and approve it” idea:

Explain every decision
Each check shows a plain-English reason, so lender and supplier both know exactly why an invoice passed or was held.

Self-dealing detector
Catches a supplier pretending to be its own buyer by comparing identity details and requiring separate buyer confirmation.

Live tamper test
A Verify chain button that turns red at the precise entry when data is altered. The most memorable moment of the demo.

Verification QR
Each approved invoice gets a QR code. Anyone can scan it to confirm the fingerprint is on record without seeing the invoice.

Risk-based advance
The advance is not a flat percentage. It moves with the buyer’s trust score and history.

Camera-first for phones
Supplier photographs the invoice in the APK and the AI fills the form. Built for shop owners, not accountants.

How we add new features without breaking the app

Treat every check as a **small plug-in function** with the same output shape. The engine runs whatever is in the list, and the screen draws any check from the same shape, so a new feature usually means writing one function and adding it to one list. No screen needs rewriting.

```
type CheckResult = { name: string; status: 'pass' | 'warn' | 'fail'; reason: string }
type Check = (invoice: Invoice) => Promise<CheckResult>

const checks: Check[] = [fileHash, duplicates, identity, poMatch, buyerRisk]
// New idea (say, 'invoice date is a weekend')? Write one function, add it here.
const results = await Promise.all(checks.map(c => c(invoice)))
```

Ideas ready to plug in later

- Round-number or just-below-threshold amounts flagged as suspicious.
- Invoice date after delivery date, or due date that is unusually long.
- Sudden jump in a supplier’s invoice volume compared with their history.
- Circular trading: A bills B, B bills C, C bills A (cycle detection on the invoice graph).
- Two or three mock lenders returning competing offers on the same invoice.

11 The 6-hour build plan

Who does what, and when it is done

Four people, but two know the system. Make the other two just as valuable by giving them the work that decides how the demo feels.

You: backend and engine
Schema, RLS, edge functions (extract, checks, audit chain, verify), trust score.

Your friend: app screens
Four role screens, upload flow, offer card, audit screen, Capacitor APK.

Teammate 3: data pack
Five invoice files, POs, receipts, five buyers with histories; checks the AI reads each file correctly.

Teammate 4: story and QA
Pitch, slides, demo script, denied-case tests, a backup screen recording of the full demo.

Time	What happens	Done when
0:00 to 0:45	Create Supabase project, tables, seed data and RLS. Wrap a blank page with Capacitor and install the APK on a real phone	APK opens on the phone and login works
0:45 to 2:00	Upload screen and the *extract-invoice* function (Gemini returns JSON)	A photo turns into fields on screen
2:00 to 3:00	Duplicate, identity and PO/receipt checks	All five sample files give the expected result
3:00 to 3:45	Buyer confirmation, trust score, decision and offer	Offer card shows reasons and advance
3:45 to 4:45	Audit chain, Verify chain screen, tamper test	Editing a row turns the chain red
4:45 to 5:30	Lender dashboard, empty and error states, cached AI results, polish	No blank screens, no crashes
5:30 to 6:00	Rehearse twice, final APK, backup recording	Demo works with Wi-Fi off

Working method in Claude Code

- Start with a lean **CLAUDE.md** (under 100 lines) and a **docs/** folder: spec, schema, AI prompts, demo script. Let the tool build from the docs.
- Review the plan before any code is written, then build **one flow at a time**.
- Write a test for each check using the five sample invoices, and test the denied cases for each role.
- Update the docs when a feature is done, so the next prompt starts from the truth.

Before the event
Free-tier limits and tool behaviour change. Create the Gemini key, the Supabase project and the Vercel project the evening before, and install Android Studio early.

12 Demo script and judge Q&A

Four minutes that tell one story

Time	Beat	What the judges see
0:00	The problem	Tell Ravi's story in 20 seconds. Name the four lender fears
0:20	Clean invoice	Phone photo → AI fields → checks go green → buyer confirms → ₹2,04,000 offer with reasons
1:10	Duplicate	Upload the same invoice again. Blocked, with the date it was first submitted
1:40	Mismatch	100 billed vs 80 delivered. Held for review with a plain-English explanation
2:10	Self-dealing	Supplier and buyer share a bank account. Blocked
2:40	Tamper test	Edit the amount in the database, press Verify chain, and it turns red at the exact entry
3:30	Wrap-up	What is real, what is mock, and the future scope. Then questions

Questions judges are likely to ask

Question	Honest answer
Is this a real blockchain?	In the prototype it is a hash chain in the database. The latest hash can be anchored on a public testnet; a production system would anchor to a chain or regulated ledger.
Could an admin rewrite the chain?	Yes, if nothing outside the database holds the head hash. That is why the head hash is shown to the auditor and can be anchored externally.
How accurate is the AI?	It only reads fields. The supplier confirms them, low confidence goes to human review, and all decisions come from deterministic rules.
Is it licensed? Does it move money?	No. It is a prototype with mock data. Real platforms in India are regulated by the RBI.
What about privacy?	Files stay in a private bucket with short-lived links. The audit log holds hashes, not documents.
How would it scale?	Each check is an independent function, Postgres is indexed on the matching fields, and heavy work can move to a queue later.

Sources

- YourStory: How small businesses can use TReDS to finance their trade receivables: https://yourstory.com/smbstory/msme-treds-trade-receivables-discounting-financing
- Oxyzo: How TReDS is changing invoice financing in India: https://oxyzo.in/blogs/how-treds-is-changing-invoice-financing-in-india
- SMFG India Credit: TReDS definition and how it works: https://www.smfgindiacredit.com/knowledge-center/trade-receivables-electronic-discounting-system-treds.aspx
- Canara Bank: Trade Receivables Discounting System: https://canarabank.com/pages/treds
- YourStory (2024): M1xchange and invoice discounting: https://yourstory.com/2024/11/invoice-discounting-amazon-backed-m1xchange-aims-to-double-business
- Anantam IAS: M1xchange and TReDS for MSMEs: https://anantamias.com/m1xchange-treds-msme/
- SYNESIS 26 Fintech problem statement 2: Invoice Financing with Hybrid Audit Architecture